```
→ ~ docker run -itd --network=host --name=demo1 ubuntu
5501af6b3edaa43513291ca01827ac495e0378bb3979ee52cbbb333fe314ad6e
→ ~
```

Figure 4: Assigning a container to a specific network

```
→ ~ docker network inspect host
[
    {
        "Name": "host",
        "Id": "5d989ebbd9c99a17760d3e91d87119ce25fe7
        "Created": "0001-01-01T00:00:00Z",
        "Scope": "local",
        "Driver": "host",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": []
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "Containers": {
            "5501af6b3edaa43513291ca01827ac495e0378b
                "Name": "demo1",
                "EndpointID": "93a4347cd47839e11e4f6
                "MacAddress": "",
                "IPv4Address": "",
                "IPv6Address": ""
```

Figure 5: Inspecting the host network

Bridged network: When a new container is launched, without the *--network* argument, Docker connects the container with the bridge network, by default. In bridged networks, all the containers in a single host can connect to each other through their IP addresses. This type of network is created when the span of Docker hosts is one, i.e., when all the containers run on a single host. To create a network that has a span of more than one Docker host, we need an overlay network.

Host network: Launching a Docker container with the *--network=host* argument pushes the container into the host network stack where the Docker daemon is running. All interfaces of the host (the system the Docker daemon is running in) are accessible from the container which is assigned the host network.

None network: Launching a Docker container with the *--network=none* argument puts the Docker container in its own network stack. So, IP addresses are not assigned to the containers in the none network, because of which they cannot communicate with each other.

So far, we have discussed the default networks created by Docker. In the next sections, we will see how users can create their own networks and define them as per their requirements.

Assigning IP addresses to Docker containers

While creating Docker containers, random IP addresses are allocated to the containers. An IP address is assigned only to running containers. A container will not be assigned any IP address if it is assigned a none network. So, use the bridge network to view the command displaying the IP address. The standard *ifconfig* command displays the networking details including the IP address of different network interfaces. Since the *ifconfig* command is not available in a container, by default, we use the following command to get the IP address. The container ID can be taken from the output of the *$ docker ps* command.

```
$docker inspect --format '{{ .NetworkSettings.IPAddress }}'
<Container ID>
```

In the above command, *--format* defines the output as *.NetworkSettings.IPAddress*. Inspecting the network or container returns all the networking information related to it. *.NetworkSettings.IPAddress* will filter out the IP address from all the networking information.

In addition, you can also look at the */etc/hosts* file to get the IP address of the Docker container. This file contains the networking information of the container, which also includes the IP address. These two ways of checking the IP address of a container are shown in Figure 6.

We can also assign any desired IP address to the Docker container. If we are creating a bunch of containers, we can specify the range of IP addresses within which these addresses of the containers should lie. This can be done by creating user-defined networks.

Creating user-defined networks

We can create our own networks called user-defined networks in Docker. These are used to define the network as per user requirements, and this dictates which containers can communicate with each other, what IP addresses and subnets should be assigned to them, etc. How to create user-defined networks is shown below.

```
$docker network create --subnet=172.18.0.0/16 demonet
```

```
→ ~ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED
d8ad4fb272d0        ubuntu              "/bin/bash"         5 seconds ago
→ ~ docker inspect --format '{{ .NetworkSettings.IPAddress }}' d8ad4fb272d0
172.17.0.2
→ ~ docker attach d8ad4fb272d0
root@d8ad4fb272d0:/#
root@d8ad4fb272d0:/# cat /etc/hosts
127.0.0.1       localhost
::1     localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
172.17.0.2      d8ad4fb272d0
root@d8ad4fb272d0:/#
```

Figure 6: Checking the IP address of a container

```
→ ~ docker network create --subnet=173.20.0.0/16 demonet
:6574473a2b787fa7b7eef6502abdca69294d6b2c2d769ea0b8183cde01c148c
```

Figure 7: Creating a user defined network